スプライン関数を用いた関数の近似

毛利海斗

目次

1 **はじめに** 2

2 **スプライン関数** 3
2.1 切断べき関数 3

3 $B-$ **スプライン関数** 4
3.1 差分商 4
3.2 $(k-1)$ 次 $B-$ スプライン関数 6
3.3 ドブァ・コックスの算法 6

4 **スプライン補間** 7
4.1 スプライン補間のプログラム 7
4.2 コードの説明 9
4.3 実行結果 11

5 **おわりに** 12

1　はじめに

「スプライン」とは製図のとき滑らかな曲線を描く道具のことである。
この道具のようにスプライン関数を用いると自在に滑らかな曲線を描くことができる。
例として以下のようなものがある。

$$f(x) = \begin{cases} 3 - 2x^2 & (x \leq -2) \\ x^3 + 4x^2 + 12x + 11 & (-2 < x \leq -1) \\ x^2 + 9x + 10 & (-1 < x \leq 1) \\ -x^3 + 4x^2 + 6x + 11 & (1 < x \leq 4) \\ 3x^3 - 44x^2 + 198x - 245 & (4 < x \leq 5) \\ 2x^3 - 29x^2 + 123x - 120 & (x > 5) \end{cases} \tag{1}$$

この $f(x)$ は六つの区間でそれぞれ異なった多項式が与えられている。
この小区間を与える座標を一般に節点 (連結点) といい、この節点はその点の左右どちらの式でもその節点を代入すれば $f(x)$ は同じ値になる。
この $f(x)$ は well-defined でかつ、単射の関数となる。

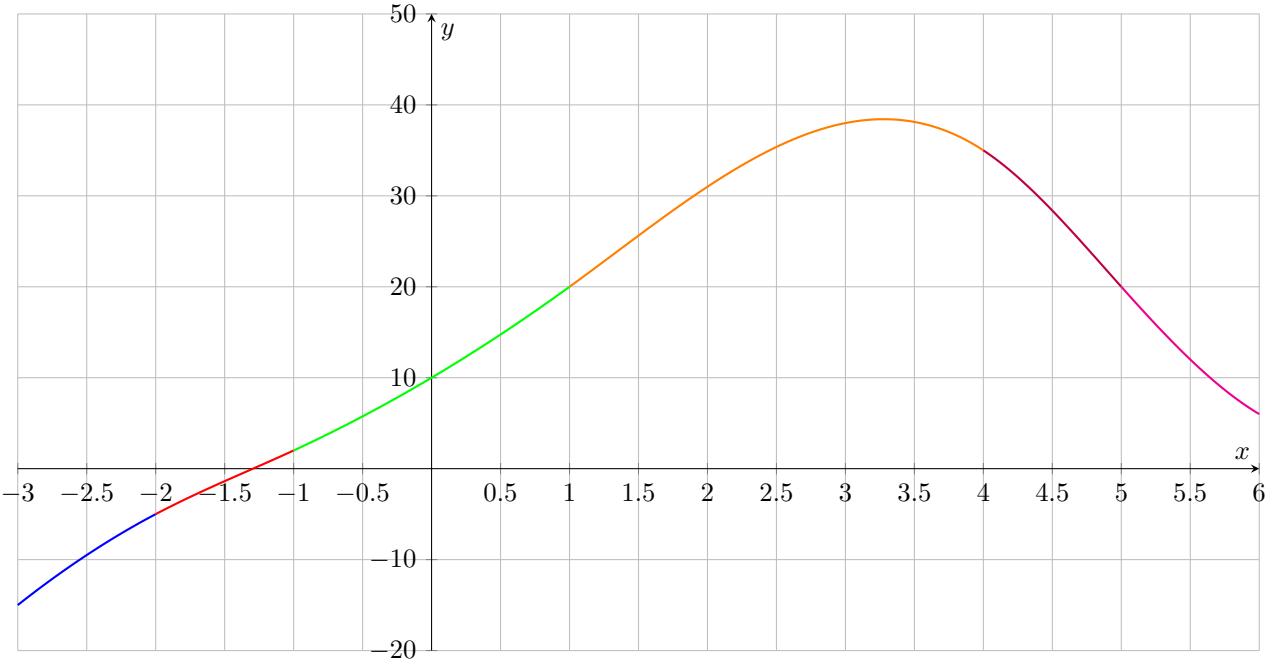


図 1　$f(x)$ のグラフ

ちなみにこの $f(x)$ は上のグラフのように滑らかになる。

2　スプライン関数

スプライン関数の定義は以下のようなものとする。

定義 1.
$q_1, q_2, \cdots, q_n$ を増加する実数列として、これをスプライン関数の節点とすると、節点 $q_1, q_2, \cdots, q_n$ から構成される m 次スプライン関数は、次の条件 (i),(ii) を満たす関数 $s(x)$ である。ただし、$m \leq n-1$ とする。

(i) 各小区間 $q_i \leq x \leq q_{i+1}$ $(i = 1, 2, \cdots, n-1)$ で $s(x)$ は m 次かまたはそれ以下の多項式 $a_0 + a_1 x + a_2 x^2 + \cdots + a_m x^m$ で与えられる。

(ii) $s(x)$ とその $1, 2, \cdots, m-1$ 階微分は、全区間 $q_1 \leq x \leq q_n$ で連続である。ただし、0 次スプライン関数はこの条件には含まない。また、1 次のスプライン関数は折れ線の関数である。

0 次スプライン関数は各小区間が定数の階段関数であるので連続ではないことがある。
ここからスプライン関数 $s(x)$ を切断べき関数を用いて表していく。

2.1　切断べき関数

x_0 で一つの節点を持ち、$x \leq x_0$ に対して常に 0 であり、$x \geq x_0$ に対してある多項式 $p_m(x)$ を与えるような m 次のスプライン関数を考える。
$s(x)$ は $x \leq x_0$ で常に 0 であるから左微分はすべて 0 になるが、定義 1 より $s(x)$ 自身と $1, 2, \cdots, m-1$ 階微分は連続でないといけないので、多項式 $p_m(x)$ とその $1, 2, \cdots, m-1$ 階微分は $x = x_0$ で m 重根を持たなければならない。
よって、それは次のようになる。

$$p_m(x) = c(x - x_0)^m \quad (x > x_0 \text{で } c \text{ はある定数}) \tag{2}$$

ここで切断べき関数 x_+ を定義する。

定義 2. 切断べき関数

$$x_+^m = \begin{cases} x^m & (x > 0) \\ 0 & (x \leqq 0) \end{cases}$$

と定義する。
これによって (2) は

$$p_m(x) = \begin{cases} c(x - x_0)^m & (x > x_0) \\ 0 & (x \leq x_0) \end{cases} \tag{3}$$

$$= c(x - x_0)_+^m \tag{4}$$

とすれば、$x \leq x_0$ に対して常に 0 であり、$x \geq x_0$ のときに多項式 $p_m(x)$ になるような式が出来上がる。
これを足し合わせることで節点が複数ある場合でも m 次スプライン関数は表すことができる。

定理 1.
$p_m(x)$ は m 次の多項式であり、c_i は定数係数であるとすると節点 $x_1, x_2, \cdots, x_n$ を持つ m 次スプライン関数 $s(x)$ は、一義的に次の式で表される。

$$s(x) = p_m(x) + \sum_{i=1}^{n} c_i (x - x_i)_+^m$$

実際、(1) の $f(x)$ はこの定理 1 のように切断べき関数を用いて表すと、

$$f(x) = 3 - 2x^2 + (x+2)_+^3 - (x+1)_+^3 - (x-1)_+^3 + 4(x-4)_+^3 - (x-5)_+^3$$

とひとつの式で表現することができる。
このように表されればこの $f(x)$ が well-defined で単射であることはわかるであろう。

3 $B-$ スプライン関数

3.1 差分商

これから $B-$ スプラインで用いる差分商を定義する。$y = f(x)$ を考える。$q_i, q_{i+1}, \cdots, q_{i+n}$ をそれぞれ異なる座標とする。
一階差分商を

$$\begin{aligned} f(q_i, q_{i+1}) &= \frac{f(q_{i+1}) - f(q_i)}{q_{i+1} - q_i} \\ &= \frac{f(q_{i+1})}{q_{i+1} - q_i} - \frac{f(q_i)}{q_{i+1} - q_i} \end{aligned}$$

と定義する。
二階差分商は

$$\begin{aligned} f(q_i, q_{i+1}, q_{i+2}) &= \frac{f(q_{i+1}, q_{i+2}) - f(q_i, q_{i+1})}{q_{i+2} - q_i} \\ &= \frac{\left(\frac{f(q_{i+2})}{q_{i+2} - q_{i+1}} - \frac{f(q_{i+1})}{q_{i+2} - q_{i+1}}\right) - \left(\frac{f(q_{i+1})}{q_{i+1} - q_i} - \frac{f(q_i)}{q_{i+1} - q_i}\right)}{q_{i+2} - q_i} \\ &= \frac{\frac{f(q_{i+2})}{q_{i+2} - q_{i+1}} - f(q_{i+1})\left(\frac{1}{q_{i+2} - q_{i+1}} - \frac{1}{q_{i+1} - q_i}\right) + \frac{f(q_i)}{q_{i+1} - q_i}}{q_{i+2} - q_i} \end{aligned}$$

ここで、

$$\begin{aligned}\frac{1}{q_{i+2}-q_{i+1}}-\frac{1}{q_{i+1}-q_i}&=\frac{(q_{i+1}-q_i)+(q_{i+2}-q_{i+1})}{(q_{i+2}-q_{i+1})(q_{i+1}-q_i)}\\&=-\frac{q_{i+2}-q_i}{(q_{i+2}-q_{i+1})(q_{i+1}-q_i)}\end{aligned}$$

よって、

$$\begin{aligned}f(q_i,q_{i+1},q_{i+2})&=\frac{\frac{f(q_{i+2})}{q_{i+2}-q_{i+1}}-f(q_{i+1})\left(\frac{1}{q_{i+2}-q_{i+1}}-\frac{1}{q_{i+1}-q_i}\right)+\frac{f(q_i)}{q_{i+1}-q_i}}{q_{i+2}-q_i}\\&=\frac{\frac{f(q_{i+2})}{q_{i+2}-q_{i+1}}+f(q_{i+1})\frac{q_{i+2}-q_i}{(q_{i+2}-q_{i+1})(q_{i+1}-q_i)}+\frac{f(q_i)}{q_{i+1}-q_i}}{q_{i+2}-q_i}\\&=\frac{f(q_i)}{(q_{i+2}-q_i)(q_{i+1}-q_i)}+\frac{f(q_{i+1})}{(q_{i+2}-q_{i+1})(q_{i+1}-q_i)}+\frac{f(q_{i+2})}{(q_{i+2}-q_{i+1})(q_{i+2}-q_i)}\end{aligned}$$

そして、高階差分商は $(r=0,1,2,\cdots,n-1)$ は、

$$\begin{aligned}f(q_i,q_{i+1},\cdots,q_{i+r+1})&=\frac{f(q_{i+1},q_{i+2},\cdots,q_{i+r+1})-f(q_i,q_{i+1},\cdots,q_{i+r})}{q_{i+r+1}-q_i}\\&=\frac{f(q_i)}{(q_i-q_{i+1})(q_i-q_{i+2})\cdots(q_i-q_{i+r+1})}+\frac{f(q_{i+1})}{(q_{i+1}-q_i)(q_{i+1}-q_{i+2})\cdots(q_{i+1}-q_{i+r+1})}\\&\quad+\cdots+\frac{f(q_{i+r+1})}{(q_{i+r+1}-q_{i+1})(q_{i+r+1}-q_{i+2})\cdots(q_{i+r+1}-q_{i+r})}\\&=\frac{f(q_i)}{p_i(q_i)}+\frac{f(q_{i+1})}{p_{i+1}(q_{i+1})}+\cdots+\frac{f(q_{i+r+1})}{p_{i+r+1}(q_{i+r+1})}\end{aligned}$$

ただし、このとき $p_j(q_j)$ は

$$p(x)=(x-q_i)(x-q_{i+1})\cdots(x-q_{i+r+1}) \tag{5}$$

$$=\prod_{l=0}^{r+1}(x-q_{i+l}) \tag{6}$$

から $(x-q_j)$ の因子を取り除き、$x=q_j$ としたもの

$$p_i(q_j)=(q_j-q_i)(q_j-q_{i+1})\cdots(q_j-q_{j-1})(q_j-q_{j+1})\cdots(q_j-q_{i+r+1}) \tag{7}$$

とする。
また、このとき $f(q_i)$ は 0 階の差分商と考えることができる。

3.2 $(k-1)$ 次 $B-$ スプライン関数

$(k-1)$ 次の $B-$ スプラインを $B_{i,k}(x)$ と書くことにし、節点 q は $q_i \leq q_{i+1} \leq q_{i+2} \leq \cdots \leq q_{i+k}$ であるとする。

定義 3.

$$B_{i,k}(x) = (q_{i+k} - q_i) M_k(x : q_i, q_{i+1}, \cdots, q_{i+k})$$

で定義される。

ここで $M_k(x : q_i, q_{i+1}, \cdots, q_{i+k})$ は $M_k(x : k) = (q-x)_+^{k-1}$ に対して $q_i, q_{i+1}, \cdots, q_{i+k}$ の節点で q に関する k 階差分商をとったものである。

また、

$$\begin{aligned} M_k(x : k) &= (q-x)_+^{k-1} \\ &= (x-q)^{k-1} - (-1)^{k-1}(x-q)_+^{k-1} \end{aligned}$$

なので定義 3 は $k = r+1$ とすると

$$\begin{aligned} B_{i,k}(x) &= (q_{i+k} - q_i) M_k(x : q_i, q_{i+1}, \cdots, q_{i+k}) \\ &= (-1)^k (q_{i+k} - q_i) \left(\frac{(x-q_i)_+^{k-1}}{p_i(q_i)} + \frac{(x-q_{i+1})_+^{k-1}}{p_{i+1}(q_{i+1})} + \cdots + \frac{(x-q_{i+k})_+^{k-1}}{p_{i+k}(q_{i+k})} \right) \end{aligned} \tag{8}$$

スプライン関数はこの $B-$ スプライン関数を重ね合わせると表すことができる。

3.3 ドブァ・コックスの算法

この (8) は解析的に解くとなったならば問題はないのだがコンピュータによって計算をしようとした場合には $(x-q_{i+k})_+^{k-1}$ が原因で桁落ちをする恐れがある。

なので、この問題を解決するドブァ・コックスの算法についてを記す。

定理 2. 一般に、$(k-1)$ 次の $B-$ スプライン関数 $B_{i,k}(x)$ は、漸化式

$$B_{i,k}(x) = \frac{x - q_i}{q_{i+k-1} - q_i} B_{i,k-1}(x) + \frac{q_{i+k} - x}{q_{i+k} - q_{i+1}} B_{i+1,k-1}(x)$$

で表すことができる。ここで、

$$\begin{aligned} B_{i,1}(x) &= -(q_{i+1} - q_i) \left(\frac{(x-q_i)_+^0}{q_i - q_{i+1}} + \frac{(x-q_{i+1})_+^0}{(q_{i+1} - q_i)} \right) \\ &= (x-q_i)_+^0 - (x-q_{i+1})_+^0 \end{aligned} \tag{9}$$

切断べき関数を用いないと

$$B_{i,1}(x) = \begin{cases} 1 & (q_i \leq x < q_{i+1}) \\ 0 & (x < q_i,\ q_{i+1} \leq x) \end{cases} \tag{10}$$

なので定理 2 を用いれば 1 次、2 次、$\cdots$、$(k-1)$ 次と漸化式なので次々と求めだすことができる。

また、$(k-1)$ 次のスプライン関数 $s(x)$ は、$B-$ スプライン $B_{i,k}(x)$ を基底とする一次結合

$$s(x) = \sum_{i=1}^{n} \alpha_i B_{i,k}(x) \tag{11}$$

で表すことができる。

4 スプライン補間

4.1 スプライン補間のプログラム

以下のプログラムはドブァ・コックスの算法を用いた $f(x) = \frac{1}{1+x^2}$ の補間プログラムである。これはC言語を用いている。

```
#include <stdio.h>
#include <math.h>

#define N 4 // データ点の個数
#define K 4 // ( k−1 ) 次スプライン
#define M 200

/* 補間したい関数( 変更可能 ) */
double f(double x){
    return 1.0 / (1.0 + x * x);
}

/* スプライン関数B B_{i,k}(xが求まる) */
double B(int i, int k, double x, double q[]){
    double left = 0.0;
    double right = 0.0;

    //if k=1
    if(k == 1){
        if(q[i] <= x && x < q[i+1])
            return 1.0;

        /* x=1 を最後だけ含める */
        if(x == q[N+K−1] && i == N−1)
            return 1.0;

        return 0.0;
    }

    /* 左項 */
    double d1 = q[i+k−1] − q[i]; // 0 になるかの判別

    if(d1 != 0.0){
        left = (x − q[i]) / d1 * B(i, k−1, x, q);
    }

    /* 右項 */
    double d2 = q[i+k] − q[i+1];

    if(d2 != 0.0){
        right = (q[i+k] − x) / d2 * B(i+1, k−1, x, q);
    }

    return left + right;
}

/* 掃き出し法 */
void gauss(double A[N][N], double b[N], double x[N]){
    int i, j, k;

    /* 前進消去 */
    for(k=0; k<N−1; k++){
        for(i=k+1; i<N; i++){
            double p = A[i][k] / A[k][k];
            /* 行をでかけて全部引くp */
            for(j=k; j<N; j++){
                A[i][j] −= p * A[k][j];
            }
            b[i] −= p * b[k];
        }
    }
```

```
    /* 後退代入 */
    for(i=N-1; i>=0; i--){
        x[i] = b[i];

        for(j=i+1; j<N; j++){
            x[i] -= A[i][j] * x[j];
        }

        x[i] /= A[i][i];
    }
}

int main(){
    int i, j;
    /* 補間点データ点 (x_n,y_n) */
    double xdata[N];
    for(i=0; i<N; i++){
        xdata[i] = (double)i / (N-1); // (double)i i を double に変更
    }

    double ydata[N];
    for(i=0; i<N; i++){
        ydata[i] = f(xdata[i]);
    }

    /* 節点 */
    double q[N+K];

    /* 左端 */
    for(i=0; i<K; i++){
        q[i] = 0.0;
    }

    /* 内側の節点 */
    int m = N - K;

    for(i=0; i<m; i++){
        q[K+i] = (double)(i+1) / (m+1);
    }

    /* 右端 */
    for(i=N; i<N+K; i++){
        q[i] = 1.0;
    }

    /* 補間条件 */
    double A[N][N];
    for(j=0; j<N; j++){
        for(i=0; i<N; i++){
            A[j][i] = B(i, K, xdata[j], q);
        }
    }

    /* 係数 alpha */
    double alpha[N];
    gauss(A, ydata, alpha);

    /* data.txt 出力 */
    FILE *fp;

    fp = fopen("data.txt", "w");

    fprintf(fp,"# x exact spline error\n");

    /* 最大誤差平均, */
    double max_error = 0.0;
    double sum_error = 0.0;
    for(j=0; j<M; j++){
        double x = (double)j / (M-1);
        double s = 0.0;

        /*s(xΣ)= alpha_i B_{i,k}(x)*/
        for(i=0; i<N; i++){
            s += alpha[i] * B(i, K, x, q);
        }

        double exact = f(x);
        double error = fabs(exact - s);
        sum_error += error;

        /* 最大誤差の更新 */
        if(error > max_error){
            max_error = error;
        }

        fprintf(fp,"%f %f %f %f\n",x,exact,s,error);
    }
    fclose(fp);
    printf誤差の平均は("%e\n",sum_error/M);
```

```
    /* 最大誤差表示 */
    printf誤差の最大値は("%e\n",max_error);
    /* gnuplot 実行 */
    FILE *gp;

    gp = popen("gnuplot -persistent", "w");
    fprintf(gp,"set grid\n");
    fprintf(gp,"set title 'B-spline interpolation'\n");
    fprintf(gp,"plot ""'data.txt' using 1:2 with lines lw 2 title 元の関数'',""'data.txt' using 1:3 with lines lw 2 title 'B-spline''
    fflush(gp);
    pclose(gp);

    return 0;
}
```

4.2 コードの説明

(できる限り直しましたがラテフにコードを貼った影響で少し表示がおかしいところがあるかもしれません)
このコードは補間したい関数を関数 f(double x) に入れると $(k-1)$ 次 (今回は $K=4$ なので 3 次スプライン) のスプライン補間が区間 $[0,1]$ でされる。そしてその結果が gnuplot でグラフ表示、最大誤差、誤差平均が出力されるようになっている。
まず、$B-$ スプラインである

$$s(x) = \sum_{i=0}^{N-1} \alpha_i B_{i,k}(x)$$

を考える。ここから main 関数に沿って仕組みの説明をする。
次に、データ点を作りたいので区間 [0,1] を N 等分する補間点 x_j を作る。

$$x_j = \frac{j}{N-1} \quad (j = 0, 1, \cdots, N-1)$$

y_j の値は

$$y_j = f(x_j) = \frac{1}{1+x_j^2}$$

であり、これでデータ点 (x_j, y_j) ができた。

次に、節点を作る。
左端を重複させたいので

$$q_0 = q_1 = \cdots = q_{K-1} = 0$$

にする。
右端の重複は

$$q_N = q_{N+1} = \cdots = q_{N+K-1} = 1$$

これらは $s(0) = y_0, s(1) = y_{N-1}$ を満たしやすくするために入れている。
また、曲線の曲がる位置になる内側の節点の数は $N-K$ である。(先の重複分を除いて)
そしてこの節点を等間隔で配置したいので

$$q_{i+K} = \frac{i+1}{m+1} \quad (i = 0, 1, \cdots, m-1)$$

これで定理 2 と (5) の式を用いて for 文で漸化式で計算するとできるので $B_{i,k}$ が求まる。
補間するための条件は

$$y_i = s(x_j) = \sum_{i=0}^{N-1} \alpha_i B_{i,k}(x_j)$$

つまり、$j = 0, \cdots, N-1$ での連立方程式なので

$$\begin{aligned}
&\alpha_0 B_{1,k}(x_1) + \alpha_1 B_{2,k}(x_1) + \cdots + \alpha_{N-1} B_{N,k}(x_1) = y_1 \\
&\alpha_0 B_{1,k}(x_2) + \alpha_1 B_{2,k}(x_2) + \cdots + \alpha_{N-1} B_{N,k}(x_2) = y_2 \\
&\qquad\vdots \qquad\qquad\qquad\qquad \vdots \\
&\alpha_0 B_{1,k}(x_{N-1}) + \alpha_1 B_{2,k}(x_{N-1}) + \cdots + \alpha_{N-1} B_{N-1,k}(x_{N-1}) = y_{N-1}
\end{aligned}$$

すなわち、

$$\begin{pmatrix} B_{0,k}(x_0) & \cdots & B_{N-1,k}(x_0) \\ B_{0,k}(x_1) & \cdots & B_{N-1,k}(x_1) \\ \vdots & & \vdots \\ B_{0,k}(x_{N-1}) & \cdots & B_{N-1,k}(x_{N-1}) \end{pmatrix} \begin{pmatrix} \alpha_0 \\ \alpha_1 \\ \vdots \\ \alpha_{N-1} \end{pmatrix} = \begin{pmatrix} y_0 \\ y_1 \\ \vdots \\ y_{N-1} \end{pmatrix}$$

あとはこれに値を代入し、掃き出し法を用いて α を求めだす。

これで α は求めだせるのでこの係数を用いて

$$s(x) = \sum_{i=0}^{N-1} \alpha_i B_{i,k}(x)$$

を計算すれば補間関数ができる。

このあとに誤差 $e(x)$ を

$$e(x) = |f(x) - s(x)|$$

として計算し、最大値を取ることでで最大誤差を出している。
また、この計算をする際に data.txt というテキストファイルを作り、その中に数値を格納することで gnuplot にそれを読みこませてグラフを作っている。

4.3 実行結果

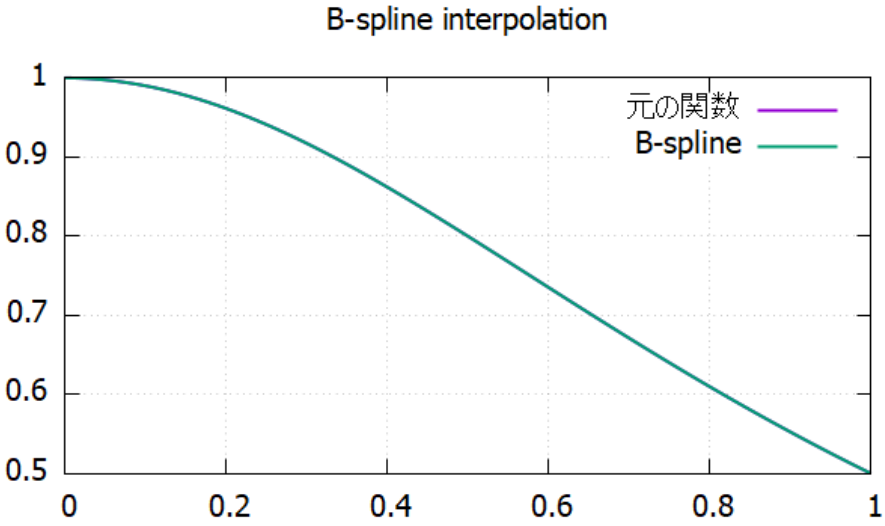


図 2　gnuplot

3 次スプライン		
データ点の数	平均誤差	最大誤差
4	1.603831×10^{-3}	3.872102×10^{-3}
8	1.929930×10^{-5}	3.986967×10^{-5}
16	3.712223×10^{-7}	2.009864×10^{-6}
32	1.408987×10^{-8}	8.457174×10^{-8}

4 次スプライン		
データ点の数	平均誤差	最大誤差
4	無し	無し
8	1.724229×10^{-5}	7.468862×10^{-5}
16	3.674697×10^{-8}	2.010134×10^{-7}
32	8.224688×10^{-10}	1.723338×10^{-8}

5 次スプライン		
データ点の数	平均誤差	最大誤差
4	無し	無し
8	1.381182×10^{-5}	5.327355×10^{-5}
16	1.183829×10^{-8}	1.292851×10^{-7}
32	3.522233×10^{-10}	9.936409×10^{-9}

この実行結果より、データ点の数が多くなればちゃんと誤差の平均と最大値が小さくなり、スプライン関数の次数が高くすれば誤差が少なるなることがわかる。

5　おわりに

今回はスプライン関数の基礎的な部分をやった。これらを応用することで微分方程式の解や解析を行うことができるので今後はそのようなことをやってみたいと思う。

参考文献

[1] 桜井明, スプライン関数入門, 東京電機大学出版局
[2] 吉村和美 高山文雄, パソコンによるスプライン関数, 東京電機大学出版局